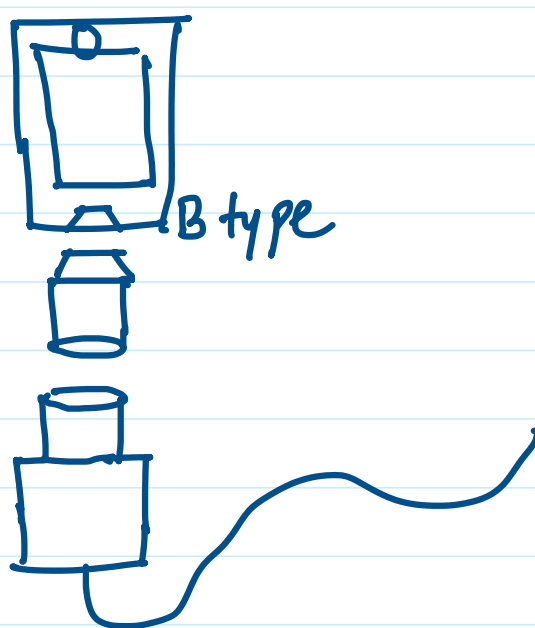


Example like :-

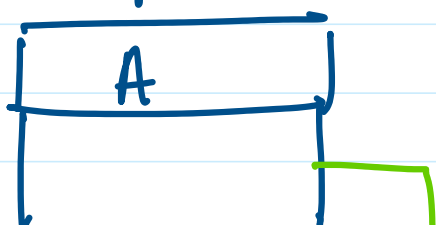
You have 'C' type charger but your phone support type 'B' so you Adapter which connect B to C type

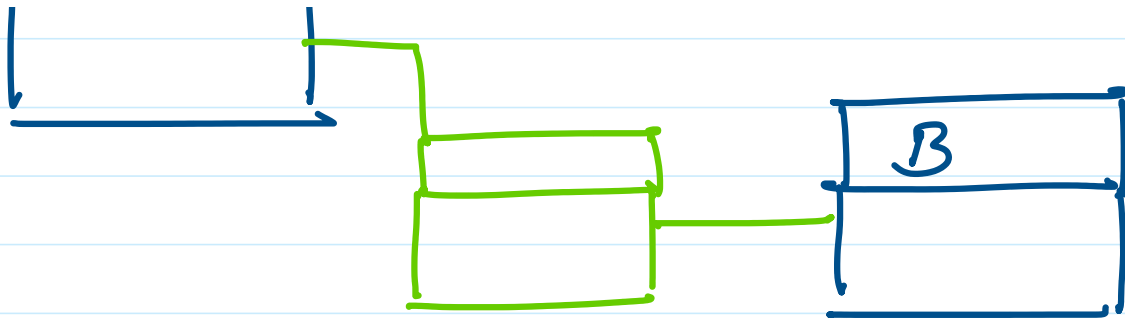


In programming

Suppose we have 2 interfaces which are totally different so they can't talk.

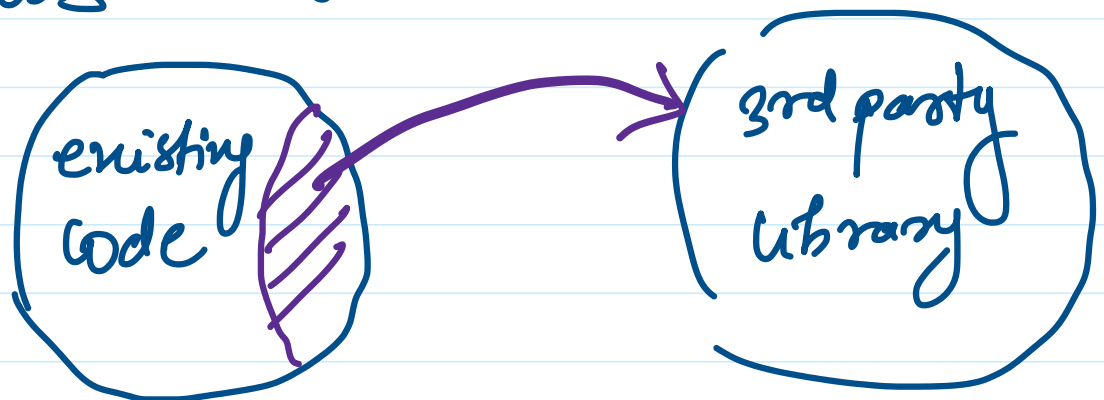
To make them talk we introduce Adaptor b/w them





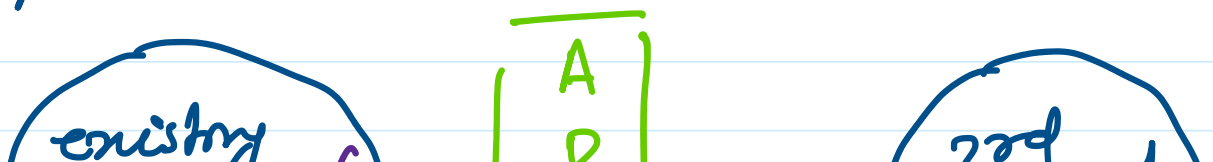
How this Pattern comes:-

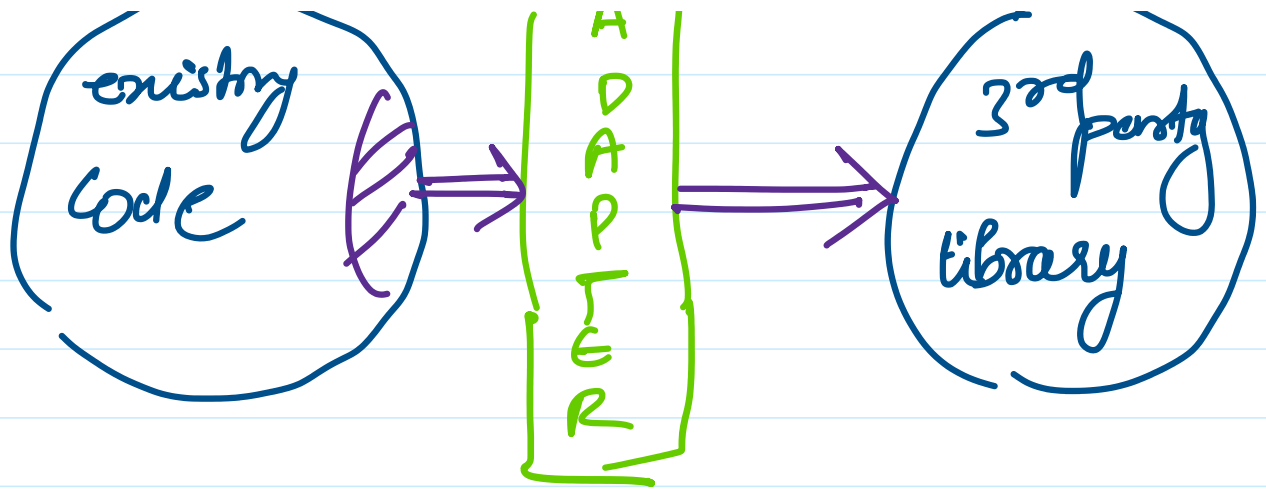
You have some existing Code and we want it to use some 3rd party library so, we reserve some space in our existing code and call 3rd party methods



So what if in future we change the 3rd party library to something else then we have to change the existing code.

So, to not do this thing we introduce Adaptor

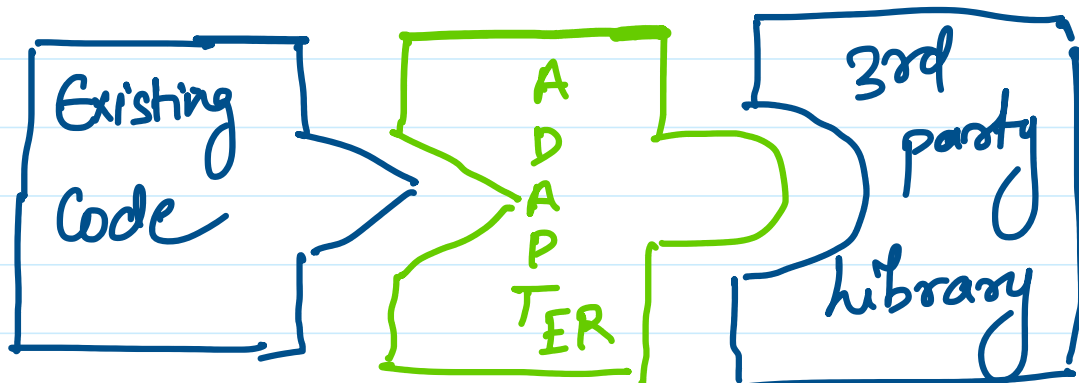




So now existing code doesn't know to which 3rd party library it is talking to
 It's a responsibility of Adaptor to interact
 Reason to Introduce Adaptor

1. Our existing code & 3rd party library might be totally different - diff. language.
2. Don't want tight coupling b/w both

So you can imagine

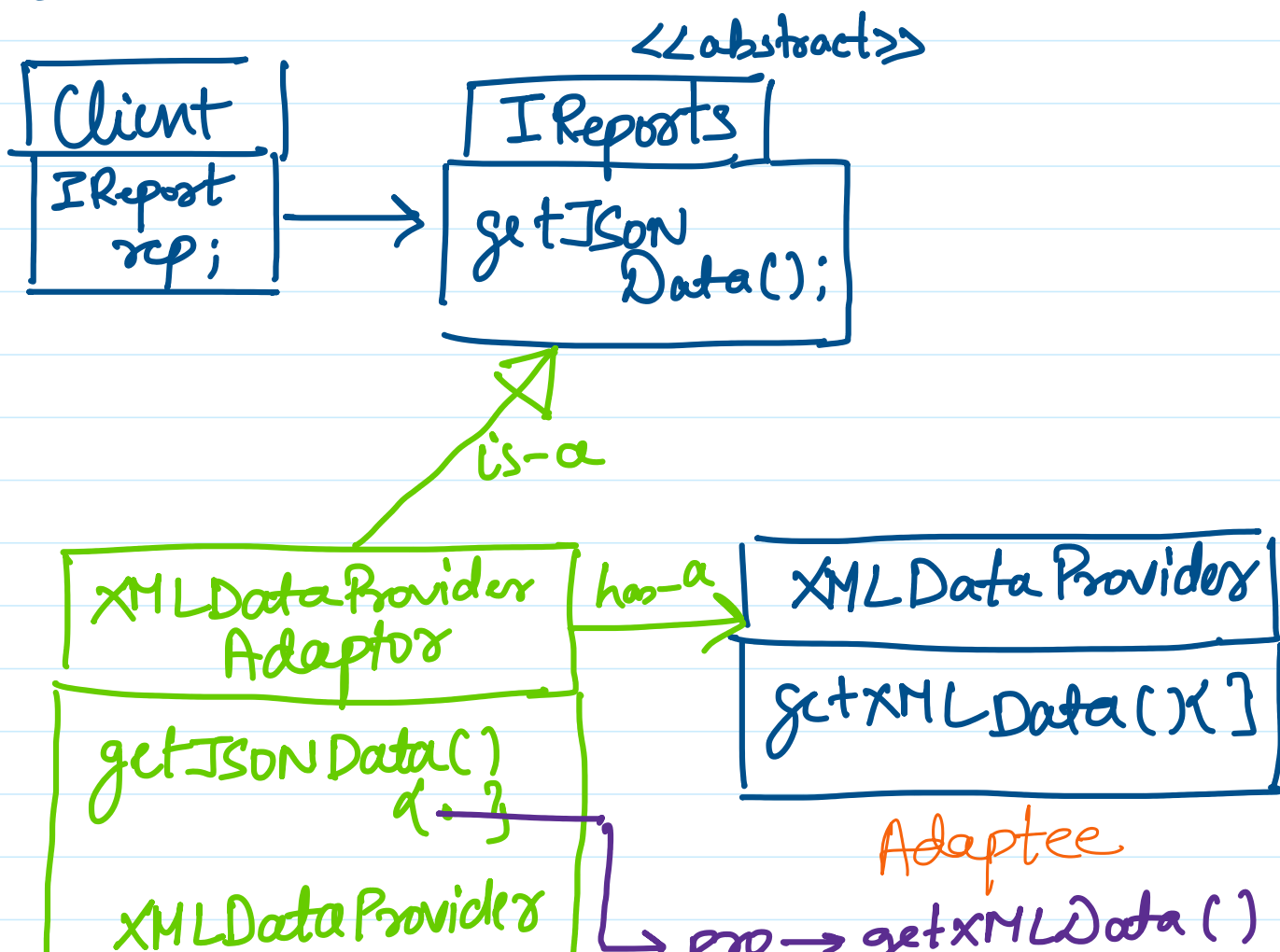


Example

Suppose we have Client which calls `getJsonData()` of Interface 'IReports' and get JSON Data.

Our task is to implement `getJsonData()`. But we have 3rd party library which gives us XML Data, we want to use it.

Our work is to convert XML to JSON



```
XMLDataProvider  
pro;  
Adapter
```

→ pro → getXMLData()
// convert into JSON
Data.

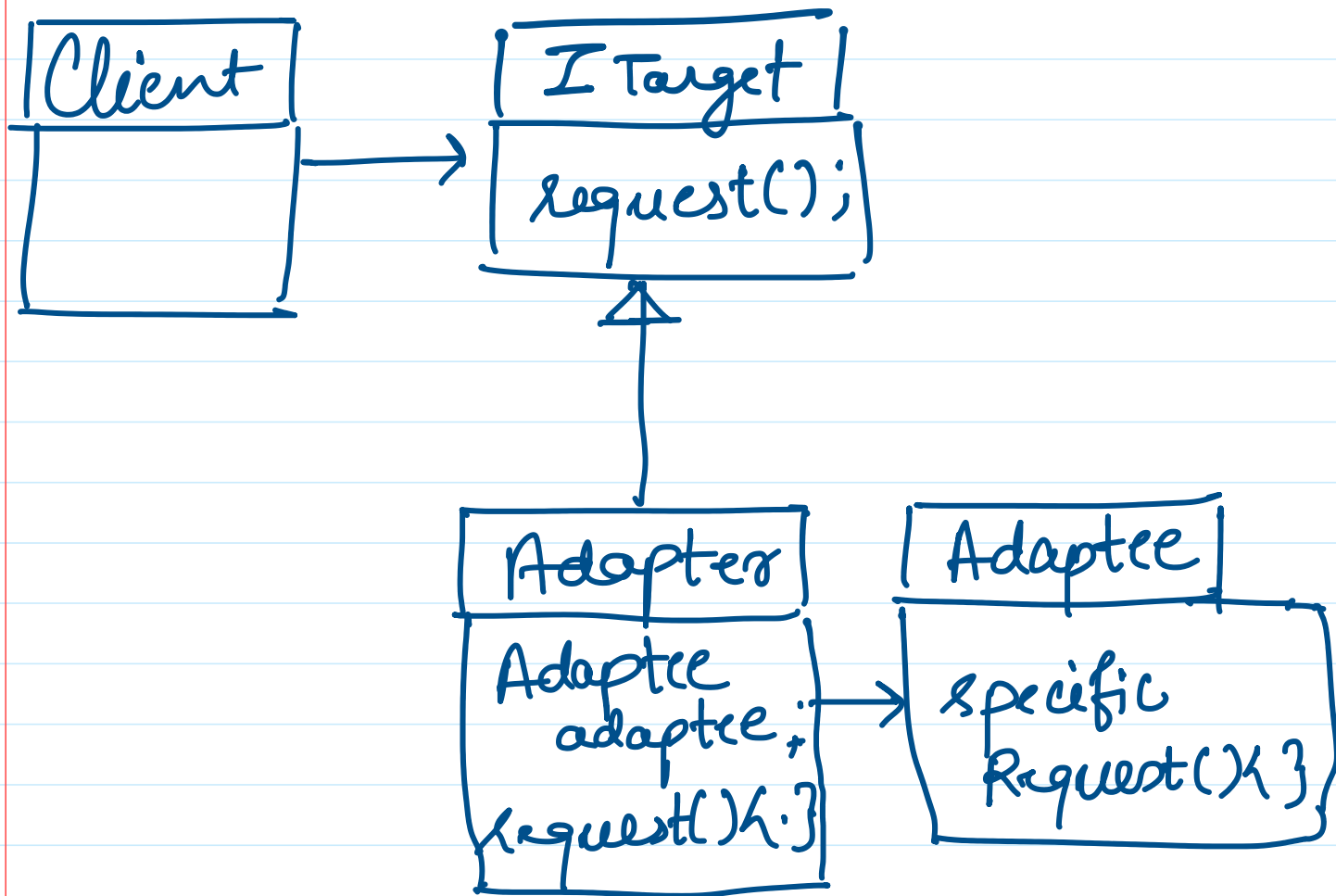
We introduce a 'Adapter' which will use XMLDataProvider and convert XML → JSON and return to Client

Definition:

Adapter Converts the Interface of a class into another interface that client expects. Adapter lets classes work together that couldn't otherwise because of incompatible interface.

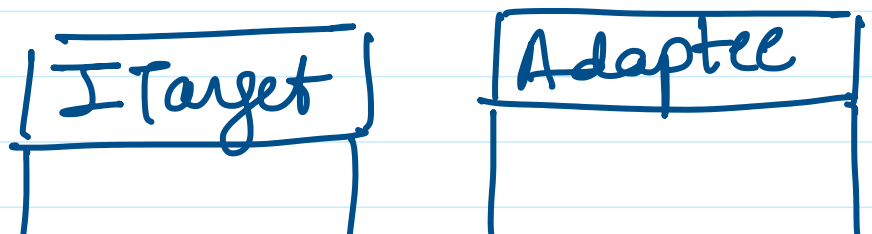
Code → see Notes.

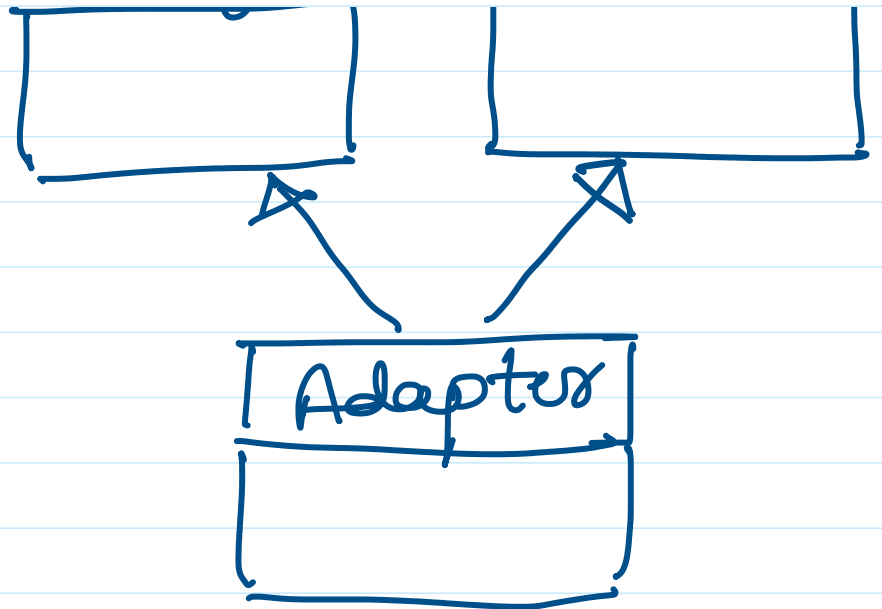
Standard UML Diagram



Above is Object Adapter

There is one more Adapter
Class Adapter





Inheritance v/s
(Class
Adapters)

X

not preferred b/c
Multiple Inheritance
is not supported

Composition
(Object
Adapters)

✓

Use Cases

- Want to integrate your existing Code with 3rd party / vendor Code
- whenever you work with legacy code . so to connect legacy code with Modern Code we use Adapter Pattern

